Winning the Space Race with Data Science

From raw launch records to geographical intelligence and predictive modelling

Eke Uka Okoro

IBM Data Science Professional Certificate • Capstone presentation

github.com/eokorouka-jpg/SpaceX-Falcon9-Data-Science-Capstone

Executive summary

90
labelled ML launches

56
Folium / Dash records

83.33%
best held-out accuracy

- End-to-end workflow: collection → cleaning → SQL / EDA → mapping → dashboard → classification
- Landing success varies across launch sites, payload bands and booster versions in the sampled data
- Logistic Regression, SVM and KNN tied on the 18-flight held-out test set

Business question & analytical objectives

Can historical launch characteristics help explain and predict successful Falcon 9 first-stage landings?

- Explore landing outcomes by site, orbit, flight history and payload
- Investigate geographical context and transportation access
- Create interactive tools for filtering and inspecting launches
- Compare tuned classifiers and evaluate prediction errors

Methodology: one connected workflow

01

COLLECT

SpaceX API + Wikipedia

02

PREPARE

clean, label, encode

03

EXPLORE

Python EDA + SQLite

04

VISUALISE

Folium + Plotly Dash

05

PREDICT

4 tuned classifiers

Python • pandas • SQL • Folium • Plotly Dash • scikit-learn

Data collection: two complementary sources

SpaceX REST API

- Request historical launch records
- Expand rocket, core, payload and pad IDs
- Flatten JSON to a structured table

Wikipedia launch tables

- Inspect HTML tables with BeautifulSoup
- Extract booster, launch and mission fields
- One saved attempt returned HTTP 403

Source note: an HTTP 403 attempt is not presented as a successful scrape.

Data wrangling | documented cleaning steps

Evidence from the API collection notebook (90 rows x 17 columns)

```
print(data_falcon9.isnull().sum())
mean_payload = data_falcon9["PayloadMass"].mean()
data_falcon9["PayloadMass"] = (
    data_falcon9["PayloadMass"]
    .fillna(mean_payload)
)
```

5 -> 0

missing PayloadMass values

LandingPad remains null when no pad was used (26 values).

Cleaned file: dataset_part_1_cleaned.csv | Source: API notebook, cells 34–35

Open the cleaning notebook on GitHub

Feature engineering | 12 features to 80 predictors

Evidence from Python EDA & feature engineering notebook

```
features_one_hot = pd.get_dummies(
    features,
    columns=["Orbit", "LaunchSite",
             "LandingPad", "Serial"]
)
features_one_hot = features_one_hot.astype("float64")
```

90 x 12

original features

90 x 80

encoded features; 0 missing

All 80 columns converted to float64; dataset_part_3.csv exported.

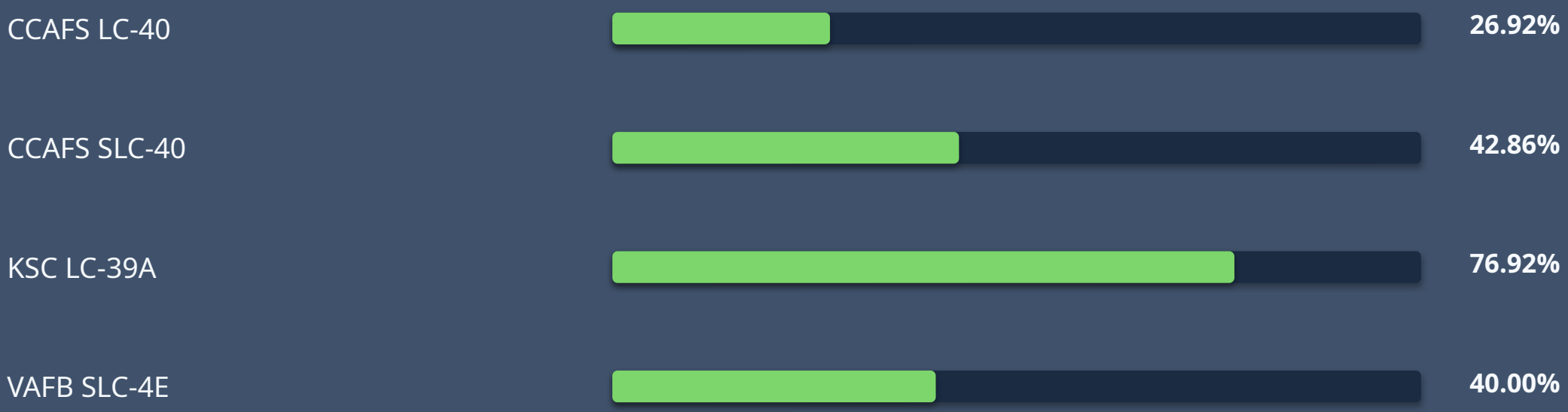
Open the feature-engineering notebook on GitHub

Why the dataset sizes differ

Analytical component	Records	Purpose
API / ML labels	90	Model training and evaluation
SQL launch table	101	Structured launch-history queries
Folium + Dash	56	Interactive geographic and dashboard analysis

These are different lab extracts and should not be combined into a single denominator.

Exploratory analysis: launch-site success



KSC LC-39A: 10 successful landings / 13 launches — strongest observed site in the 56-row sample.

EDA result: launch volume & landing outcomes

Launch-site label	Launches	Successful	Success rate
CCAFS LC-40	26	7	26.92%
CCAFS SLC-40	7	3	42.86%
KSC LC-39A	13	10	76.92%
VAFB SLC-4E	10	4	40.00%
TOTAL	56	24	42.86%

Data quality: CCAFS LC-40 and CCAFS SLC-40 are near-duplicate location labels; treat carefully.

EDA: payload mass & booster generations

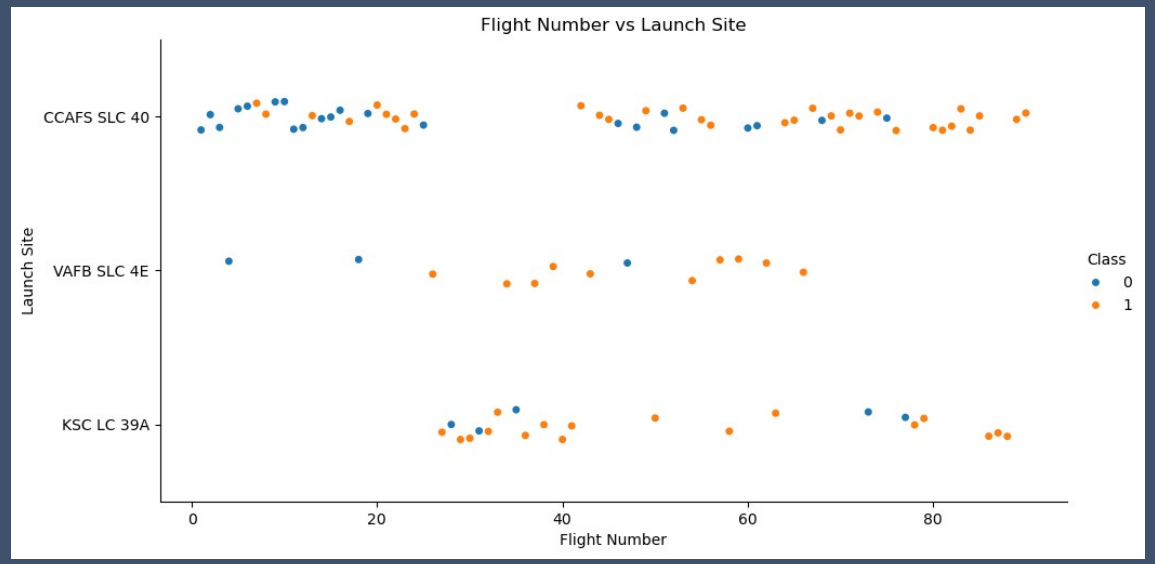


Highest observed payload-band success: 2,000–4,000 kg (13 / 21 launches).

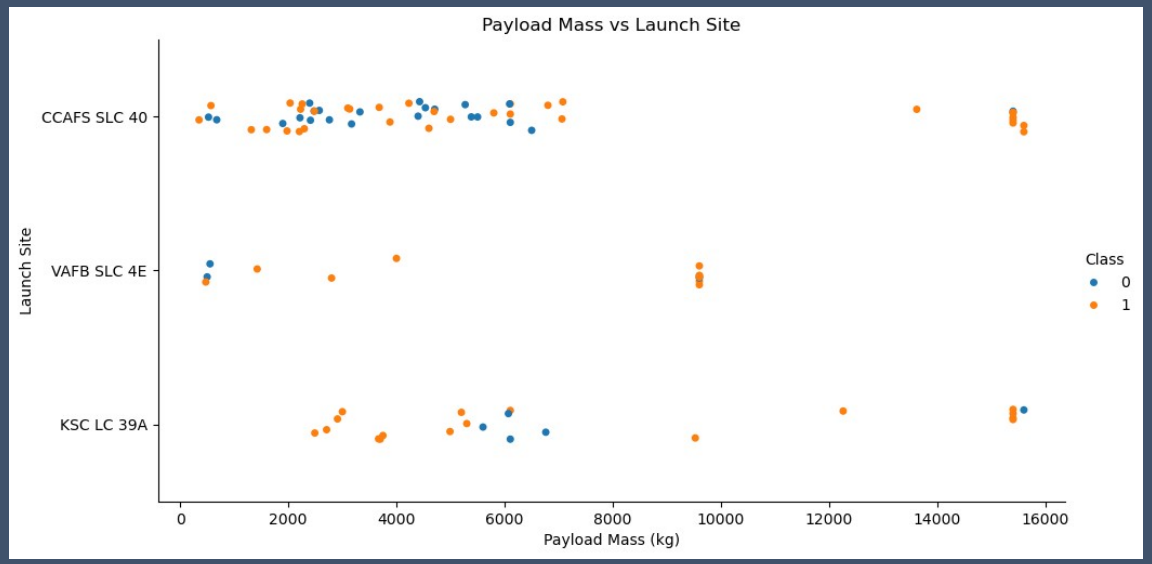
Small groups and historical confounding limit causal interpretation.

Python EDA | launch site & payload evidence

Flight number vs launch site



Payload mass vs launch site

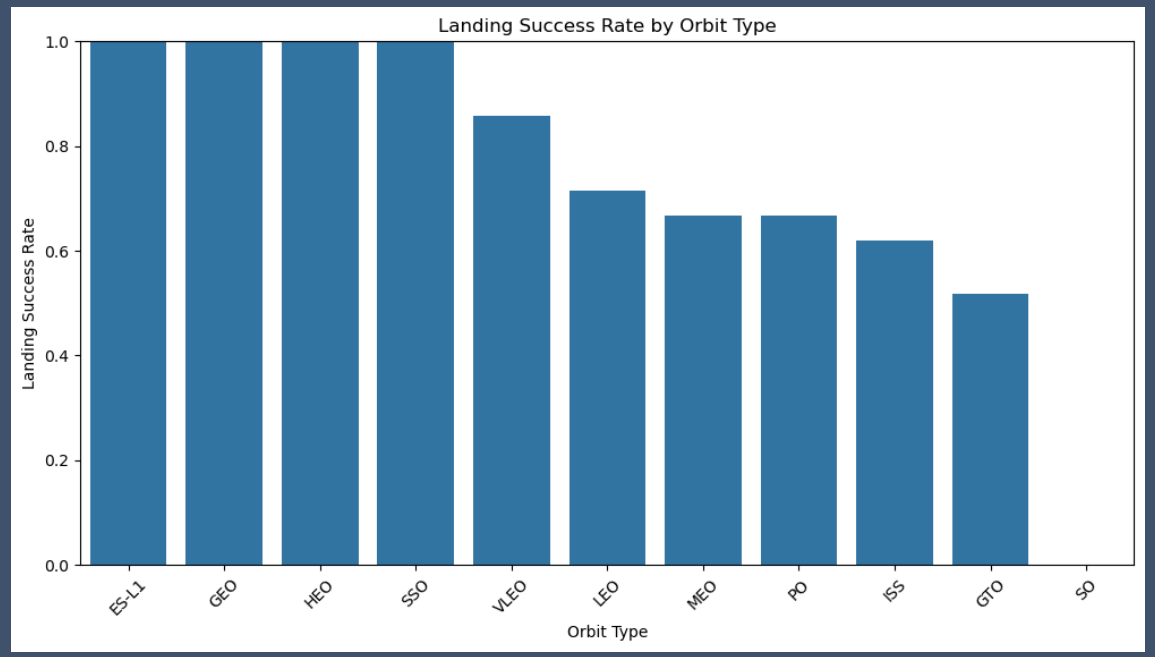


Notebook charts colour landing class (0 = unsuccessful, 1 = successful).

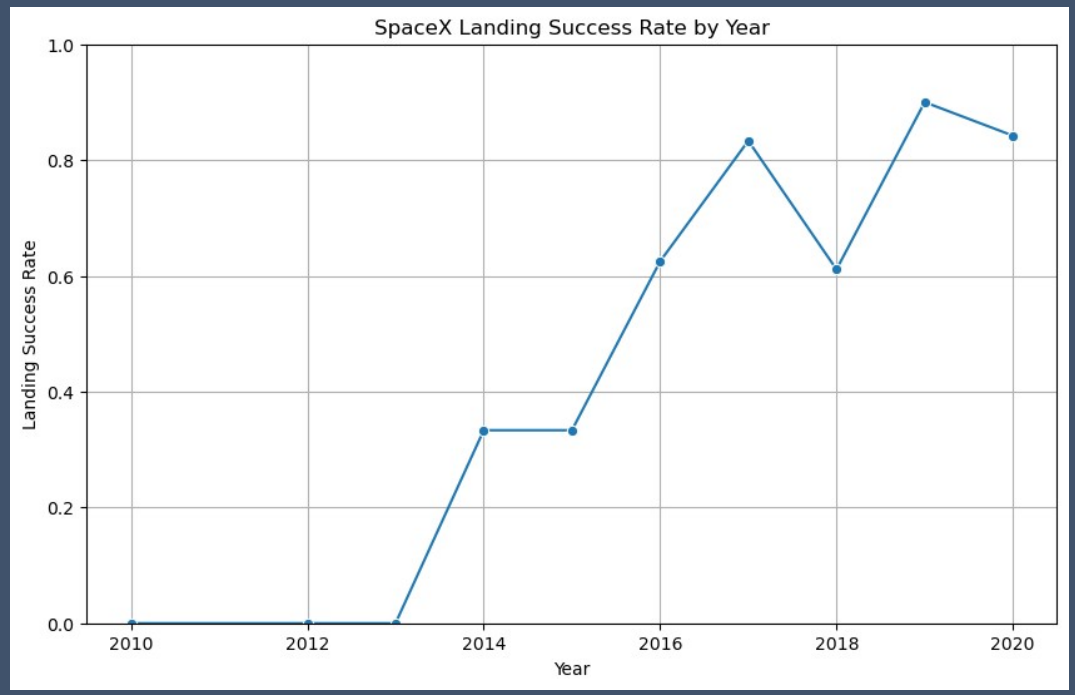
[View the original Python EDA notebook on GitHub]

Python EDA | orbit & yearly patterns

Orbit-level landing success



Landing success by launch year



Descriptive associations only: these charts do not establish causality.

View the original Python EDA notebook on GitHub

SQL analysis: queries answered with evidence

101
launch records in SQL

45,596 kg
NASA CRS total payload

2,928.4 kg
F9 v1.1 average payload

- Earliest successful ground-pad landing: 22 December 2015
- SQL distinguished mission success from first-stage landing outcomes
- Date text was converted from DD-MM-YYYY to ISO order for filtering

SQL evidence | payload mass queries

TASK 3 • NASA CRS payload

```
SELECT SUM(PAYLOAD_MASS__KG_) AS Total_Payload_Mass
FROM SPACEXTABLE
WHERE Customer = 'NASA (CRS)';
```

45,596 kg

Sum of payload masses for NASA CRS missions

TASK 4 • Falcon 9 v1.1 payload

```
SELECT AVG(PAYLOAD_MASS__KG_) AS Average_Payload_Mass
FROM SPACEXTABLE
WHERE Booster_Version = 'F9 v1.1';
```

2,928.4 kg

Mean payload mass for the specified booster version

SOURCE • Executed SQL cells and stored output tables in the SQL notebook (click to open)

SQL evidence | first ground-pad landing

TASK 5 • Find earliest successful ground-pad landing

```
SELECT MIN(
    SUBSTR(Date, 7, 4) || '-' ||
    SUBSTR(Date, 4, 2) || '-' ||
    SUBSTR(Date, 1, 2)
) AS First_Successful_Landing
FROM SPACEXTABLE
WHERE "Landing _Outcome" = 'Success (ground pad)';
```

QUERY OUTPUT

2015-12-22

SOURCE • Date strings converted to ISO format before taking MIN() (click to open notebook)

SQL output: landing outcomes, 2010–2017

Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Period: 4 Jun 2010 – 20 Mar 2017 • 31 records

SQL findings: booster and landing history

Drone-ship landing attempts

5 / 10

successful in the 2010–2017 filtered period

Maximum-payload booster IDs

12

distinct IDs; all labelled F9 B5 in the SQL results

Two failed drone-ship landings in 2015 were identified in January and April.

Folium evidence | proximity calculations & map

TASK 3 • Haversine distance, map markers and connecting lines

```
# Reference: CCAFS LC-40
launch_site = (28.562302, -80.577356)
railway_point = (28.572277, -80.653919)
highway_point = (28.493048, -80.674388)

railway_distance = calculate_distance(
    launch_site, railway_point)
highway_distance = calculate_distance(
    launch_site, highway_point)

folium.PolyLine(
    locations=[launch_site, highway_point],
    color="orange", weight=3,
    popup=f"Highway: {highway_distance:.2f} km"
).add_to(site_map)
```

Measured examples

Coastline	0.58 km
Railway	7.56 km
Highway	12.21 km
City point	18.19 km

Distances are straight-line to selected points, not road distances or proven nearest features.

Open the original Folium notebook (Task 3) on GitHub

Folium evidence | launch sites & landing outcomes

TASK 1 + TASK 2 • Interactive site markers and success/failure clusters

```
# Label each launch site on the map
for _, site in launch_sites.iterrows():
    coordinate = [site["Lat"], site["Long"]]
    folium.Circle(location=coordinate, radius=1000,
                  color="#d35400", fill=True).add_to(site_map)

# Encode landing outcome and cluster launch records
spacex_df["marker_color"] = spacex_df["class"].apply(
    lambda outcome: "green" if outcome == 1 else "red")
marker_cluster = MarkerCluster().add_to(site_map)
for _, record in spacex_df.iterrows():
    folium.Marker([record["Lat"], record["Long"]],
        icon=folium.Icon(color=record["marker_color"])).add_to(marker_cluster)
```

Interactive notebook output: click clusters to expand launches; green = landing success, red = failure.

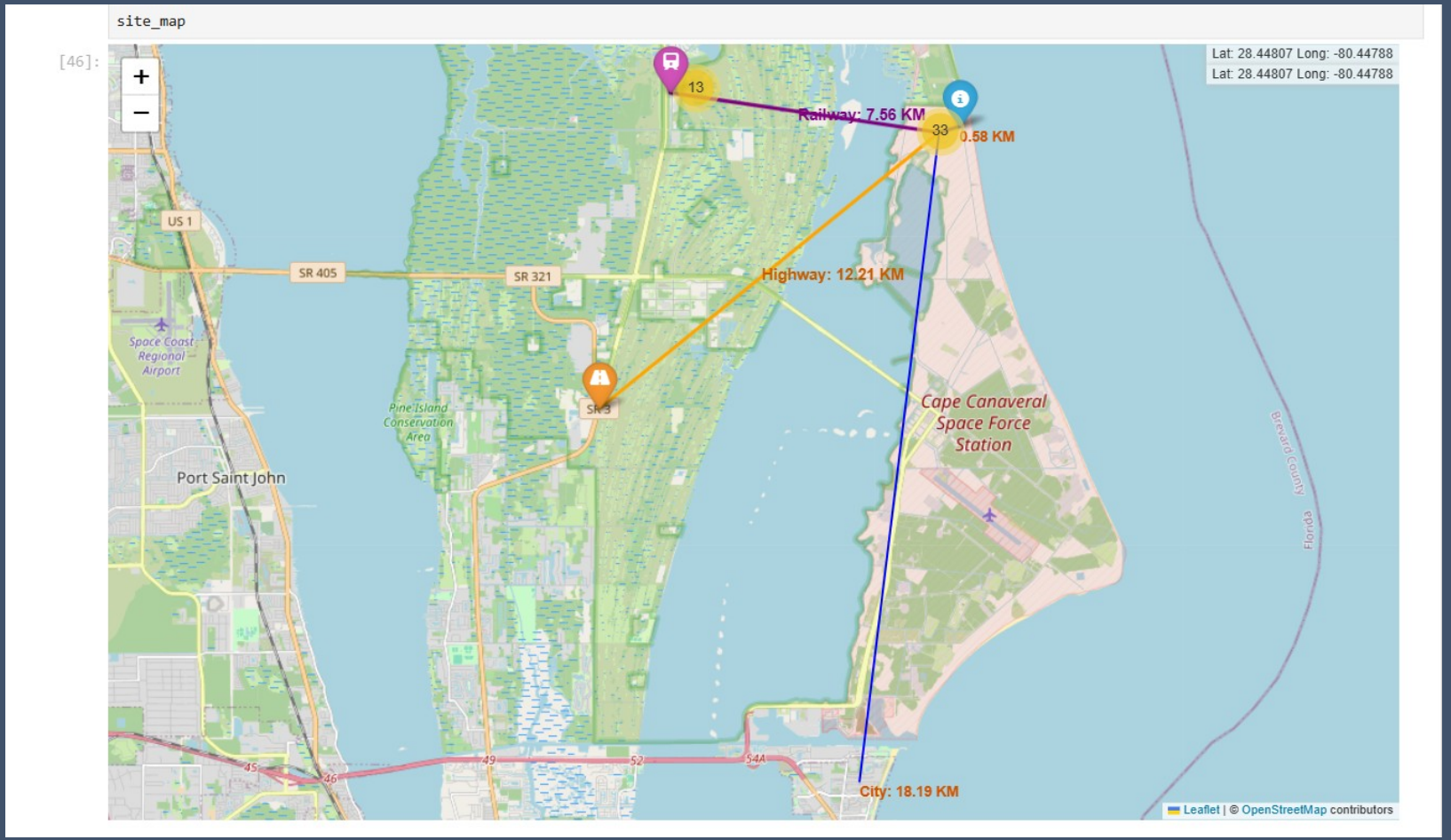
Open the original Folium notebook (Tasks 1–2) on GitHub

Folium: interactive geographical exploration

- Launch-site coordinates plotted with circles and labels
- Green / red landing-outcome markers and MarkerCluster for exploration
- MousePosition and Haversine straight-line distances
- Lines connect LC-40 to selected coast, rail, highway and city points

Map evidence: screenshot captured from the completed Jupyter Folium notebook.

Folium results: LC-40 proximity map



Measured to selected points, not verified nearest features • Coast 0.58 km • Rail 7.56 km • Highway 12.21 km • City 18.19 km

Geographic interpretation & limitations

Selected geographical point	Distance from LC-40
Coastline	0.58 km
NASA railway point	7.56 km
Florida SR 3 highway point	12.21 km
City reference point	18.19 km

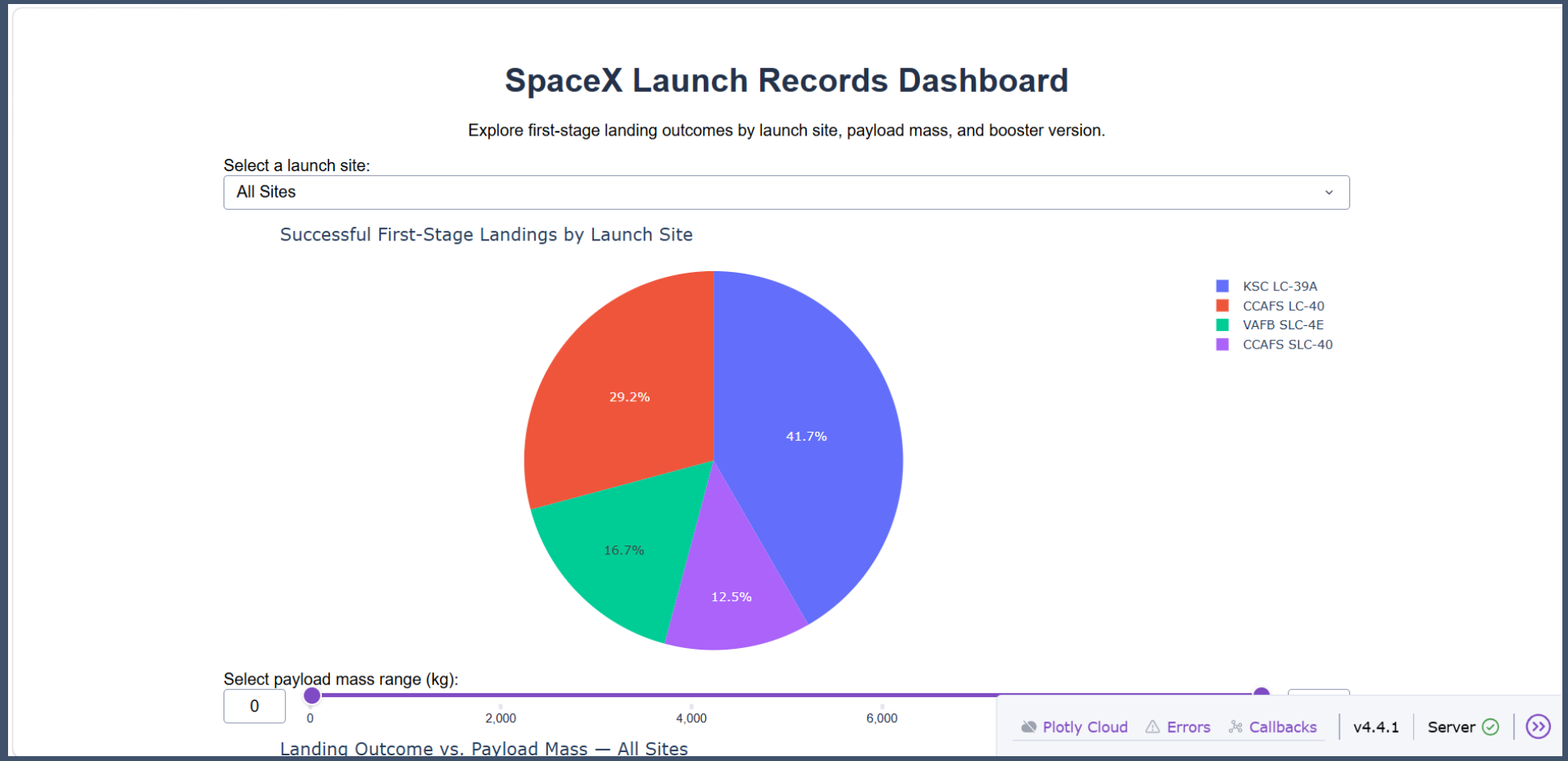
Coastal placement may facilitate trajectories over water; transport links may support logistics. These distances do not prove site-selection causes or nearest-feature proximity.

Plotly Dash: an interactive analytics app

- Launch-site dropdown: all sites or a selected location
- Pie chart: successful versus unsuccessful landings
- Payload-range slider: interactively filter launches
- Scatter plot: payload mass × outcome, coloured by booster version

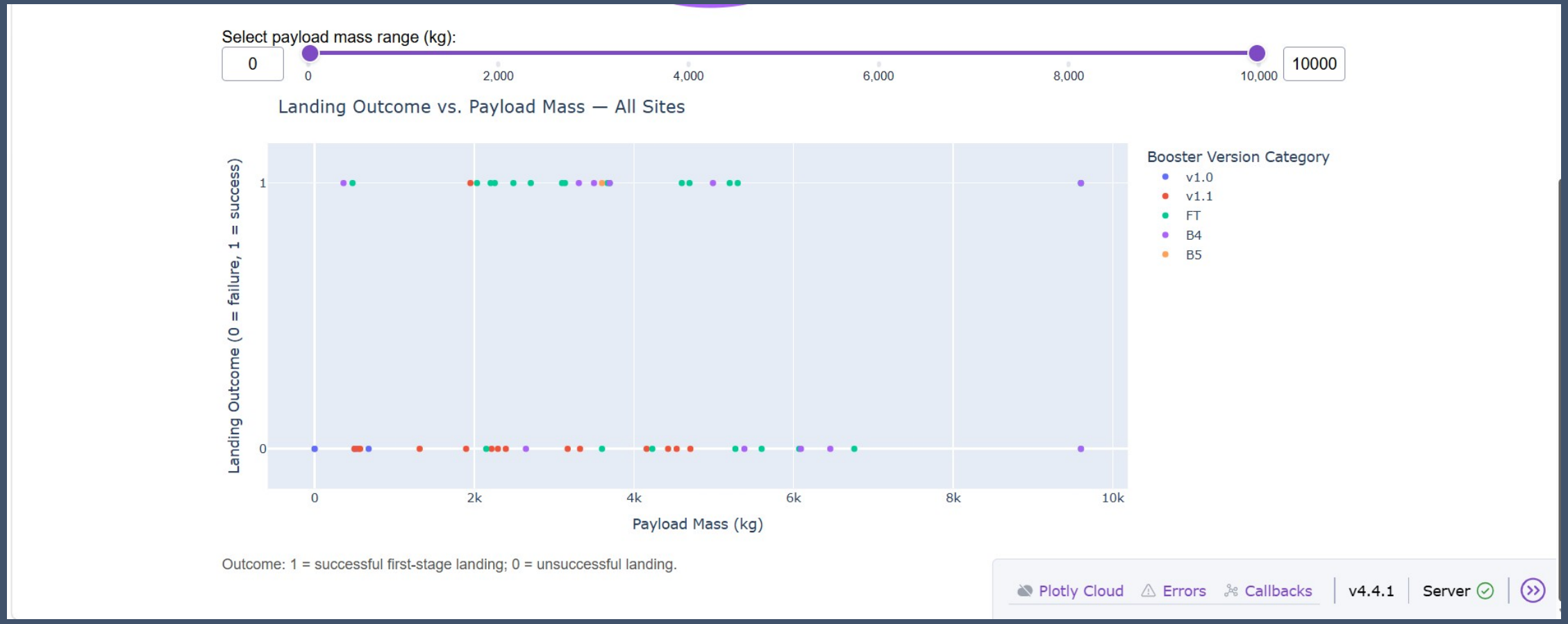
Working locally at http://127.0.0.1:8050/ • source code included in GitHub repository

Dashboard result | successful landings by site



Of 24 successful landings, KSC LC-39A contributes 10 (41.7%). Its site-specific success rate is 10/13 = 76.92%.

Dashboard result | payload and booster version



Interactive scatter: each point represents a launch; 1 = successful landing, 0 = unsuccessful. Colours distinguish booster versions.

Dashboard insights: what changed with filters

76.92%
KSC LC-39A success

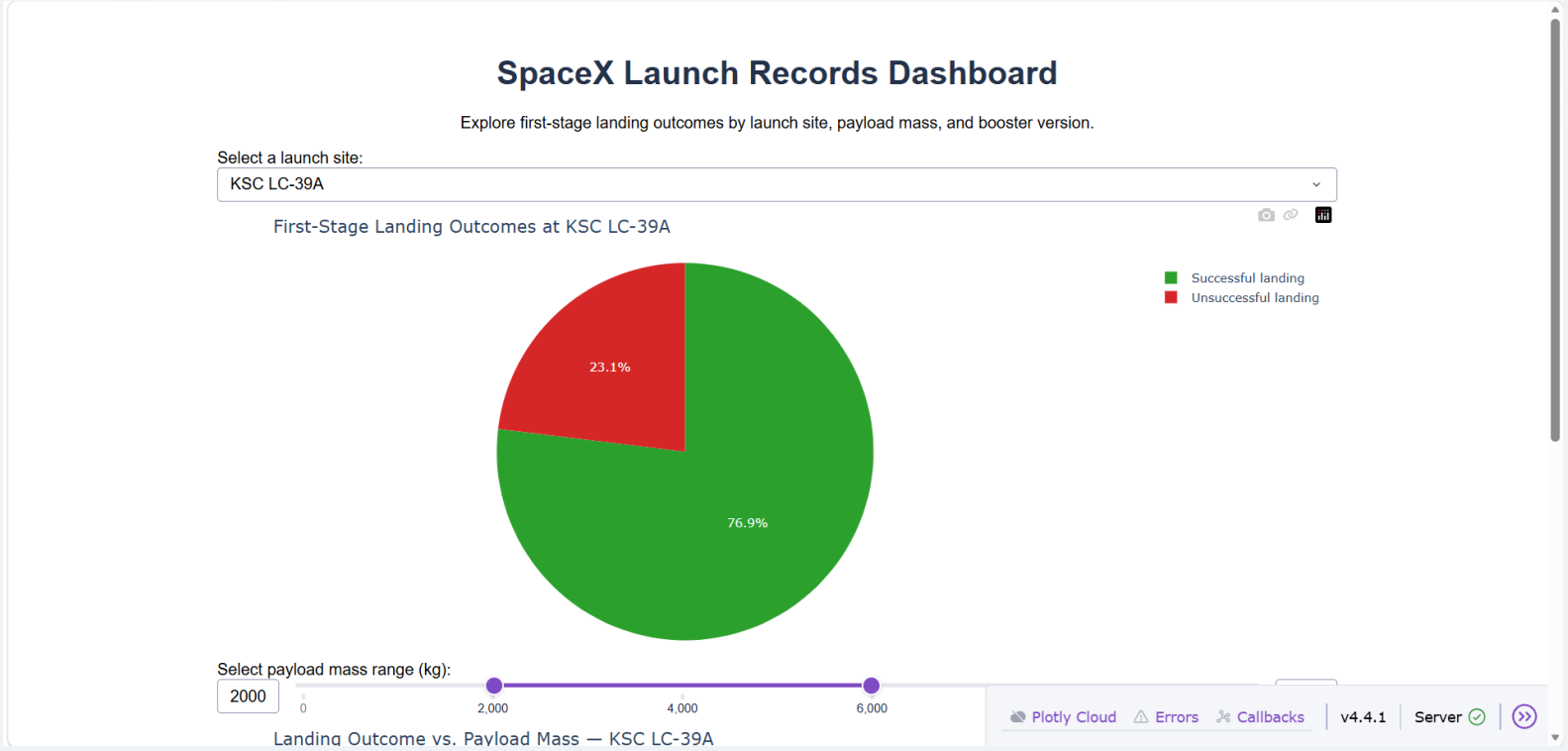
61.90%
2–4 tonne payload band

66.67%
FT booster version

- KSC LC-39A: 10 successful of 13 sampled launches
- 2,000–4,000 kg: 13 successful of 21 launches
- FT boosters: 16 successful of 24 launches; B5 had only one record

Dash evidence | site dropdown changes landing outcomes

Interactive control: launch-site dropdown set to KSC LC-39A



SELECTED SITE

KSC LC-39A

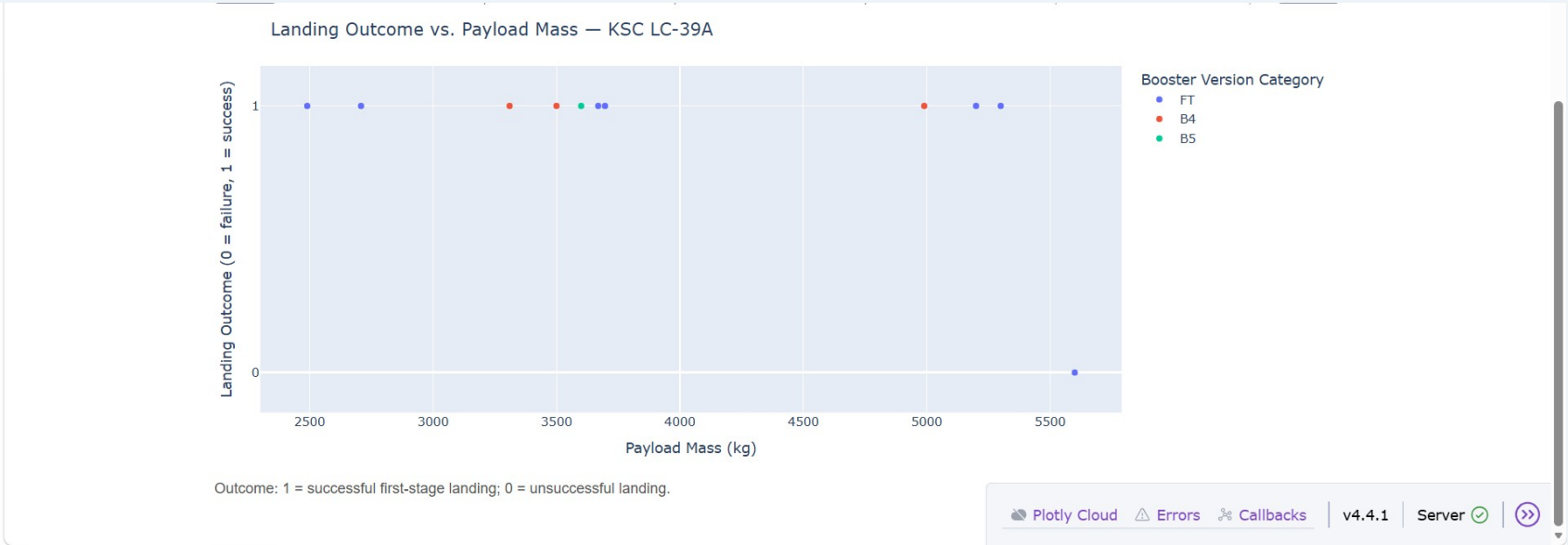
76.9%

Successful landings
10 of 13 launches

23.1% unsuccessful
(3 of 13)

Dash evidence | payload slider & booster scatter

Interactive controls: KSC LC-39A selected; payload slider filtered to 2,000–6,000 kg



FILTERS

Launch site
KSC LC-39A

Payload mass
2,000–6,000 kg

Colour = booster version
Y-axis: 1 success, 0 failure

Machine learning: modelling pipeline

01

FEATURES

80 encoded predictors

02

SPLIT

80% train / 20% test

03

TUNE

GridSearchCV; 10 folds

04

COMPARE

LR, SVM, Tree, KNN

Evaluation: 18 held-out launches; accuracy plus confusion matrices.

ML evidence | splitting and tuning classifiers

Notebook code and modelling design: 90 labelled launches, 80 encoded predictors

```
# Train/test partition and model search

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=2
)

logreg_cv = GridSearchCV(lr, parameters, cv=10)
logreg_cv.fit(X_train, Y_train)
```

TEST SET

18 / 90

20% held out for testing

10-fold cross-validation
for model selection

Open machine-learning notebook on GitHub

ML evidence | model accuracy and error types

Held-out test performance (18 records) and interpretation of confusion matrices

MODEL	TEST ACCURACY
Logistic Regression	83.33%
Support Vector Machine	83.33%
K-Nearest Neighbours	83.33%
Decision Tree	77.78%

TIED BEST MODELS

15 / 18

Correct test predictions

Confusion matrix (LR / SVM / KNN)

TN 3 • FP 3
FN 0 • TP 12

Results summary: three models tied; no unique winner.

Classification results: held-out accuracy



Three classifiers tied at 83.33% (15 correct out of 18). No unique winner on this test split.

Confusion matrices: what models missed

Model	TN	FP	FN	TP	Test accuracy
Logistic Regression	3	3	0	12	83.33%
SVM	3	3	0	12	83.33%
Decision Tree	3	3	1	11	77.78%
KNN	3	3	0	12	83.33%

For the tied models, all 12 actual positive cases were detected, but three negative cases were classified positive.

Key insights: evidence, not assumptions

- Landing success differs substantially by launch-site label in the 56-launch sample
- Payload and booster-generation patterns vary, but group sizes and time trends matter
- LC-40 is coastal; selected road and railway points are within ~12 km straight-line distance
- Model test accuracy reached 83.33%, but the hold-out contains only 18 launches
- Observed associations do not establish that geography or payload caused landing outcomes

Conclusions & practical next steps

- Delivered a reproducible capstone spanning acquisition, SQL, EDA, mapping, Dash and ML
- Use the dashboard and Folium map to explore patterns rather than infer causality
- Validate on later launch periods and larger datasets; examine class imbalance and calibration
- Standardise launch-site names and verify nearest geographic features before stronger claims
- Retrain with leakage-safe preprocessing inside cross-validation pipelines

Portfolio, sources & reproducibility

GitHub repository

github.com/eokorouka-jpg/SpaceX-Falcon9-Data-Science-Capstone

- Notebooks: API, scraping, SQL, Python EDA, Folium and classification
- Dash application and supporting CSV datasets
- Interactive map: spacex_folium_final_map.html (save and commit if available)
- Presentation figures are from the project data; Folium map is a notebook screenshot

Source context: IBM Data Science Professional Certificate capstone labs; SpaceX API and Wikipedia launch data.